

Homework 1

I.D. 330371063

File operations

Performance Analysis:

3.

Buffer size	1	64	512	1024	8192	65536
Runtime	258m*	2m 28s	0m 21s	0m 11s	0m 7s	0m 1.4s

Runtime vs. buffer size table:

*I ran the program for 31m24s and got copied 12 MB to output file, therefore the expected total time would be $100:12 \cdot 31m24s \approx 258m$

4. There is a noticeable difference in runtime between different buffer sizes. According to the measurements, we can see that the runtime decreases as buffersize increases. It happens, because a smaller buffer requires more iterations of the process, meaning more accesses to the input file and to the output file. The contents of the files are stored in different areas of memory, therefore more accesses to each file cause more accesses to the memory, and therefore longer runtime. However, when we store more characters in the buffer, we need less accesses to the input file, therefore less runtime.

5. ChatGPT answer:

what the O_EXCL option for open does

`O_EXCL` is used together with `O_CREAT` to make creation atomic.

Meaning:

```
c
int fd = open("file.txt", O_CREAT | O_EXCL | O_WRONLY, 0644);
```

Copy code

Effect

- If "file.txt" does not exist → it will be created and `open` succeeds
- If "file.txt" already exists → `open` fails and `errno` becomes `EEXIST`

So `O_EXCL` means: fail if the file already exists

Why? Mainly to prevent race conditions.

For example, two processes both want to create a new file — you don't want both to detect "file doesn't exist, let's create it", because then one could overwrite the other.

Common use: secure temp files:

```
c
int fd = open("/tmp/mytempXXXXXX",
              O_CREAT | O_EXCL | O_RDWR,
              0600);
```

Copy code

One more detail — for `O_EXCL` to mean anything, you *must* use `O_CREAT` in the same call.

`O_EXCL` alone does nothing.

Official documentation:

O_EXCL Ensure that this call creates the file: if this flag is specified in conjunction with **O_CREAT**, and `pathname` already exists, then `open()` fails with the error `EEXIST`.

When these two flags are specified, symbolic links are not followed: if `pathname` is a symbolic link, then `open()` fails regardless of where the symbolic link points.

In general, the behavior of **O_EXCL** is undefined if it is used without **O_CREAT**. There is one exception: on Linux 2.6 and later, **O_EXCL** can be used without **O_CREAT** if `pathname` refers to a block device. If the block device is in use by the system (e.g., mounted), `open()` fails with the error `EBUSY`.

On NFS, **O_EXCL** is supported only when using NFSv3 or later on kernel 2.6 or later. In NFS environments where **O_EXCL** support is not provided, programs that rely on it for performing locking tasks will contain a race condition. Portable programs that want to perform atomic file locking using a lockfile, and need to avoid reliance on NFS support for **O_EXCL**, can create a unique file on the same filesystem (e.g., incorporating hostname and PID), and use `link(2)` to make a link to the lockfile. If `link(2)` returns 0, the lock is successful. Otherwise, use `stat(2)` on the unique file to check if its link count has increased to 2, in which case the lock is also successful.

O_EXCL flag when used with **O_CREAT** flag as an argument for `open` system call ensures that the file doesn't already exist before creating it. Without **O_EXCL**, **O_CREAT** will create a new file if it doesn't exist, and will have no effect if it exists. However, **O_EXCL** will raise an error if the file already exists. According to ChatGPT, **O_EXCL** alone does nothing. On the other hand, according to the official documentation, **O_EXCL** without **O_CREAT** has undefined behaviour.

When we run `man open` it is the same as `man 1 open`, meaning the first section of the manual is shown. On the other hand, when we run `man 2 open`, the second section of the manual is shown. The first section is devoted to shell commands, and the second to system calls.

AI Experiment

Prompt: "write a program in C that copies a file's content another file"

Gemini 2.5 Pro: writes code that asks user to enter file paths and uses `fopen`, `fclose`, `fputc`. The code compiles without errors and warnings.

Prompt: "you can use only system calls. The program should get an input file, an output file and buffer size as arguments"

Gemini 2.5 Pro: satisfies all the above, but the code doesn't raise an error if the output file already exists. The code compiles without errors and warnings.

Prompt: "print an error message and exit if the output file exists"

Gemini 2.5 Pro: adds `O_EXCL` flag. The final code compiles without errors and warnings, works correctly when tested, handles errors correctly.